

Ethical Hacking — Practical Lab Reference

Practicals 1, 2 & 3 · Quick Walkthrough

P1: OWASP Juice Shop — SQL Injection & XSS

Aim: Identify and exploit two OWASP Top 10 vulnerabilities — SQL Injection and Cross-Site Scripting — in a deliberately vulnerable web app.

Tools: Docker Desktop (Windows), OWASP Juice Shop, web browser.

Step 1 — Start Juice Shop

Install Docker Desktop, open it, then in any terminal run:

```
docker run -d -p 3000:3000 bkimminich/juice-shop
```

Open browser at **http://localhost:3000**. The homepage with juice products loads.

Step 2 — SQL Injection (Login Bypass)

Click **Account** → **Login**. In the email field type the payload below; password can be anything.

```
' OR 1=1 --
```

Click **Log in** — you are now logged in as *admin*. The single quote breaks out of the SQL string and **OR 1=1** makes the WHERE clause always true, returning the first row in the users table.

Step 3 — Reflected XSS

In the top search bar, paste:

```
<iframe src="javascript:alert('xss')">
```

Press Enter — a JavaScript alert pops up, proving the search input is rendered as HTML without escaping.

Result

Successfully bypassed authentication via SQL injection and triggered a reflected XSS payload, demonstrating two OWASP Top 10 vulnerabilities in OWASP Juice Shop.

P2: Network Scanning & Reconnaissance

Aim: Inspect local network configuration, scan for open ports, and analyse captured network traffic.

Tools: Windows Command Prompt, Kali Linux (WSL), Nmap, Wireshark.

Setup (one-time)

In Administrator PowerShell:

```
wsl --install -d kali-linux
```

Reboot, create Kali username + password. Install Wireshark on Windows from [wireshark.org](https://www.wireshark.org).

Step 1 — Check network configuration

Windows CMD (Win+R → cmd):

```
ipconfig
```

Note your IPv4 address, subnet mask and default gateway.

Kali terminal:

```
ip a
```

You will see **lo** (loopback, 127.0.0.1) and **eth0** (WSL virtual NIC).

Step 2 — Test connectivity

```
ping 8.8.8.8
```

Wait ~10 lines, then press **Ctrl + C** to stop. Low, consistent round-trip times = good connection.

Step 3 — Nmap port scanning

```
sudo apt install nmap -y
nmap 127.0.0.1
nmap -p- 127.0.0.1
```

The first scan checks the 1000 most common ports; **-p-** scans all 65,535 ports. Open ports appear in the PORT / STATE / SERVICE table.

Step 4 — Wireshark packet analysis

Download a sample capture (e.g. *http.cap*) from wiki.wireshark.org/SampleCaptures. Open it in Wireshark via **File** → **Open**. Type filters in the green filter bar:

```
http tcp dns ip.addr == 127.0.0.1
```

Click any packet to see the 3-pane view (list / protocol tree / hex). Right-click an HTTP packet → **Follow** → **TCP Stream** to read the full request/response.

Result

Successfully inspected network configuration on Windows and Kali, verified internet connectivity, discovered open ports on localhost using Nmap, and analysed packet-level details (TCP / HTTP / DNS) from a sample capture in Wireshark.

P3: Password Cracking using SHA-256

Aim: Generate a SHA-256 hash of a sample password and recover the plaintext using a dictionary attack with John the Ripper.

Tools: Kali Linux (WSL), John the Ripper, rockyou.txt wordlist.

Step 1 — Install John the Ripper

```
sudo apt update
sudo apt install john -y
```

Step 2 — Generate the SHA-256 hash

```
echo -n "password123" | sha256sum
```

Output (the long hex string before the dash is the hash):

```
ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

Save just the hash to a file with **nano hash.txt** — paste only the hash (no quotes, filename, or trailing newline). Save with **Ctrl + O** → **Enter**, exit with **Ctrl + X**.

Critical: The **-n** in **echo -n** prevents a hidden newline from being included in the hash. Without it, the hash won't match anything in the wordlist.

Step 3 — Prepare the wordlist

```
sudo apt install wordlists -y
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
```

rockyou.txt is a leaked password list (~14 million entries, ~134 MB). If gzip says *no such file*, it's already extracted — move on.

Step 4 — Crack the hash

```
john --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

John hashes every word in the list with SHA-256 and compares. When it matches, it prints the plaintext (e.g. **password123**).

Step 5 — View the cracked password

```
john --show --format=raw-sha256 hash.txt
```

Expected output: **?:password123** followed by **1 password hash cracked, 0 left**.

Result

Successfully generated a SHA-256 hash of *password123*, saved it as *hash.txt*, and recovered the original password via a dictionary attack against rockyou.txt — demonstrating why unsalted SHA-256 is unsuitable for password storage. Proper alternatives: **bcrypt**, **scrypt**, **Argon2** with per-user random salts.